Ministry of Higher Education
and Scientific Research
University of Tunis
Tunis Business School

وزارة التعليم العالي و البحث
العلمي
جتمعة تونس
المعهد العالي للأعمال بتونس

INFORMATION SECURITY PROJECT 8

Website Scraper

Course: Information Assurance and Security

(IT 360)

Institution: Tunis Business School

Prepared by:
Alaa Azouzi
Sarra Alioua
Nouha Boukhris
Mariem Chammem

Supervised by:
Dr.Manel abdelkader

Academic Year: 2025–2026

Table of Contents

Table of Contents..... 2
1. Introduction..... 2
1.1. Company Introduction..... 2
1.2. Project Mission..... 3
1.3. Tools and Software Used..... 3
2. Project Conception (System Design)..... 3
2.1. System Overview..... 3
2.2. Component Breakdown..... 4
2.3. Data flow diagram..... 4
2.4. Exchange messages & data..... 6
2.5. System Architecture..... 7
3. Execution Process..... 9
3.1. Data Collection & Extraction..... 9
3.2. Data Cleaning & Transformation..... 11
3.3. Data Visualization & Insights..... 14
4. Conclusion..... 16

1. Introduction

1.1. Company Introduction

RemoteOK is a global remote job platform that aggregates job opportunities for developers, designers, data professionals, and digital workers. It is widely used by startups and tech companies to publish remote-first positions.

Unlike traditional job boards, RemoteOK provides a **public API** that exposes structured job data, making it suitable for data analysis, automation, and job market research projects.

1.2. Project Mission

The objective of this project is to design and implement a **job data extraction pipeline** that:

- Collects remote job listings from a structured API
- Filters jobs based on relevant criteria (e.g., API-related roles, tech stack, keywords)
- Cleans and organizes the data into a structured dataset
- Exports results for further analysis (CSV format)

The broader goal is to simulate a real-world **data engineering workflow** for job market intelligence.

1.3. Tools and Software Used

Figure 1: Python Logo

Figure 2: GitHub Logo

Figure 3: Draw Logo

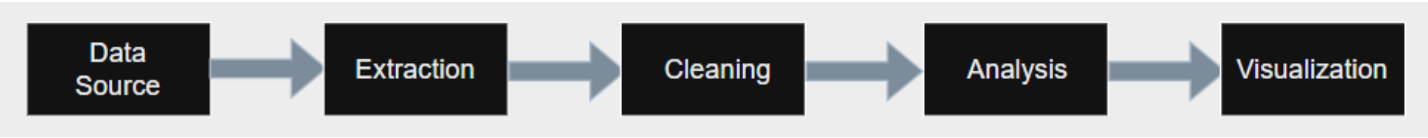
Figure 4: Power BI Logo

2. Project Conception (System Design)

2.1. System Overview

The project is designed as a **job market data pipeline system** that automatically collects, processes, and visualizes remote job listings from an external API.

The system follows a structured data engineering workflow:



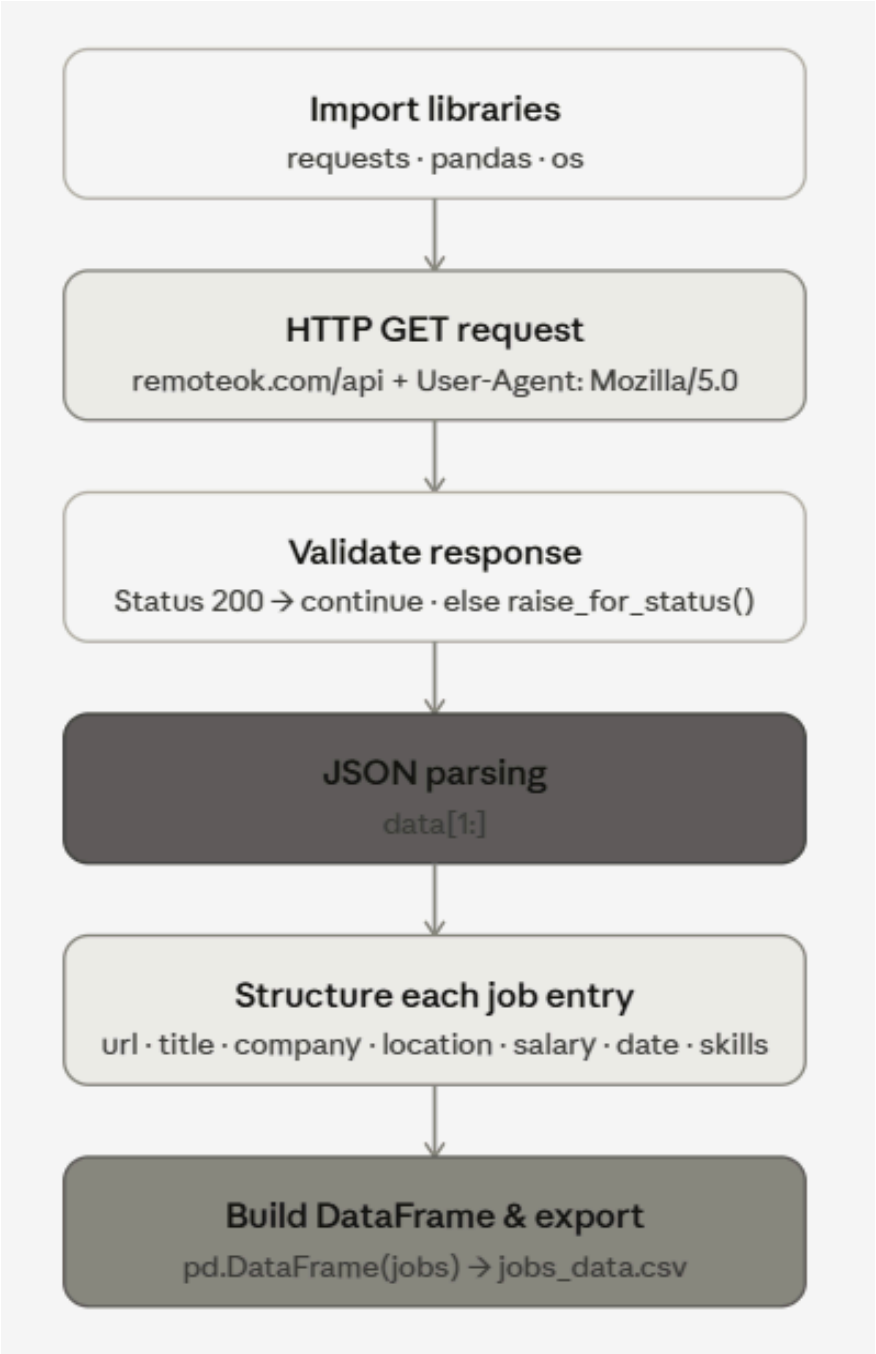
2.2. Component Breakdown

The system is composed of five main components, each with a clearly defined responsibility:

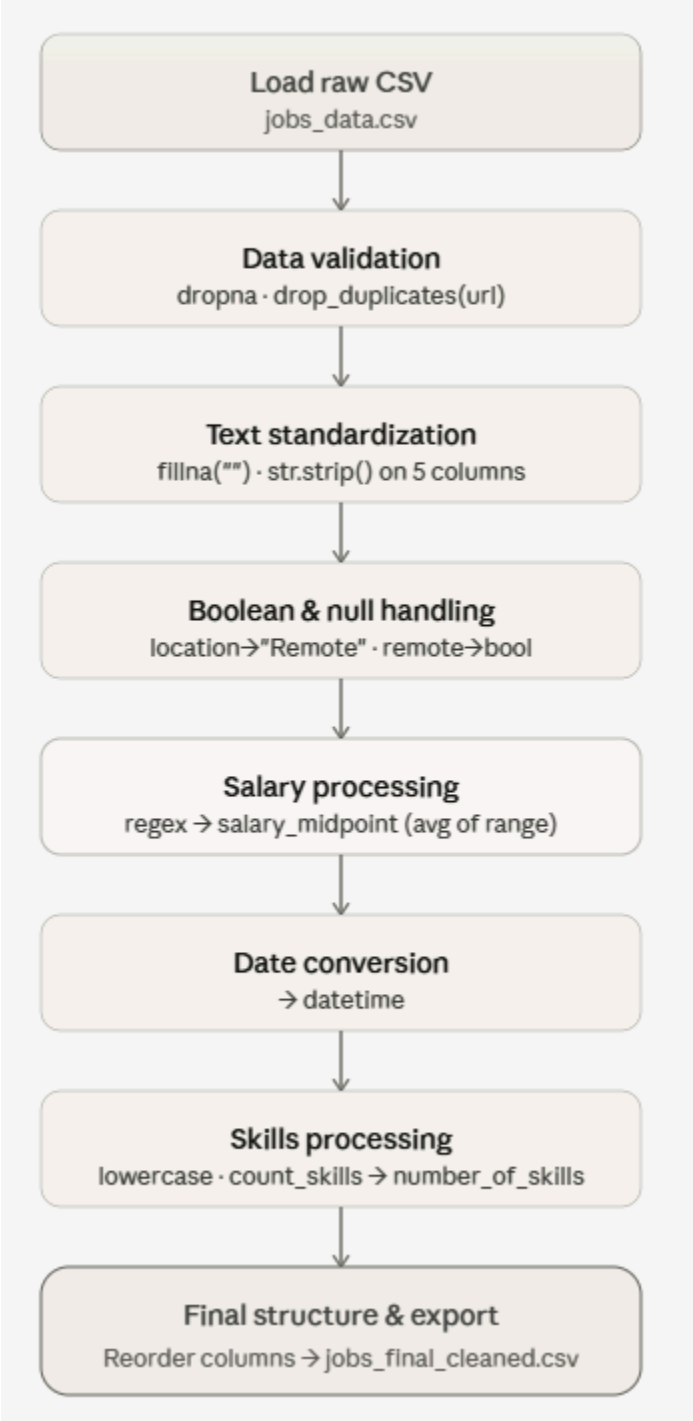
Component	File / Tool	Responsibility
API Client	remoteok_scraper.py	Sends an HTTP GET request to https://remoteok.com/api with a Mozilla/5.0 User-Agent header. Receives a JSON array of job objects. Validates the HTTP status code (must be 200) before proceeding.
JSON Parser / Data Extractor	remoteok_scraper.py	Iterates over all job objects (skipping the first metadata element). Extracts the fields: url, title, company, location, salary, date_posted, and skills (tags). Stores each job as a Python dictionary in a list.
DataFrame Builder	remoteok_scraper.py	Converts the list of dictionaries into a pandas DataFrame. Prints a preview (df.head()) and shape to the console. Saves the raw output as jobs_data.csv.
Data Cleaning Module	remoteok_cleaner.py	Loads jobs_data.csv. Drops rows missing both title and company. Removes duplicate entries by URL. Standardises text columns (strip whitespace). Replaces empty location with 'Remote'. Cleans salary strings and extracts a numeric midpoint (salary_midpoint). Converts date_posted to datetime. Counts skills per job (number_of_skills). Saves the result as jobs_final_cleaned.csv.
Visualisation Layer	Power BI / Excel	Loads jobs_final_cleaned.csv. Produces dashboards covering: top hiring companies, most demanded skills, salary distribution, job type breakdown, and posting trends over time.

2.3. Data flow diagram

The diagrams below illustrate how data moves through extraction phase :



The diagrams below illustrate how data moves through cleaning phase :



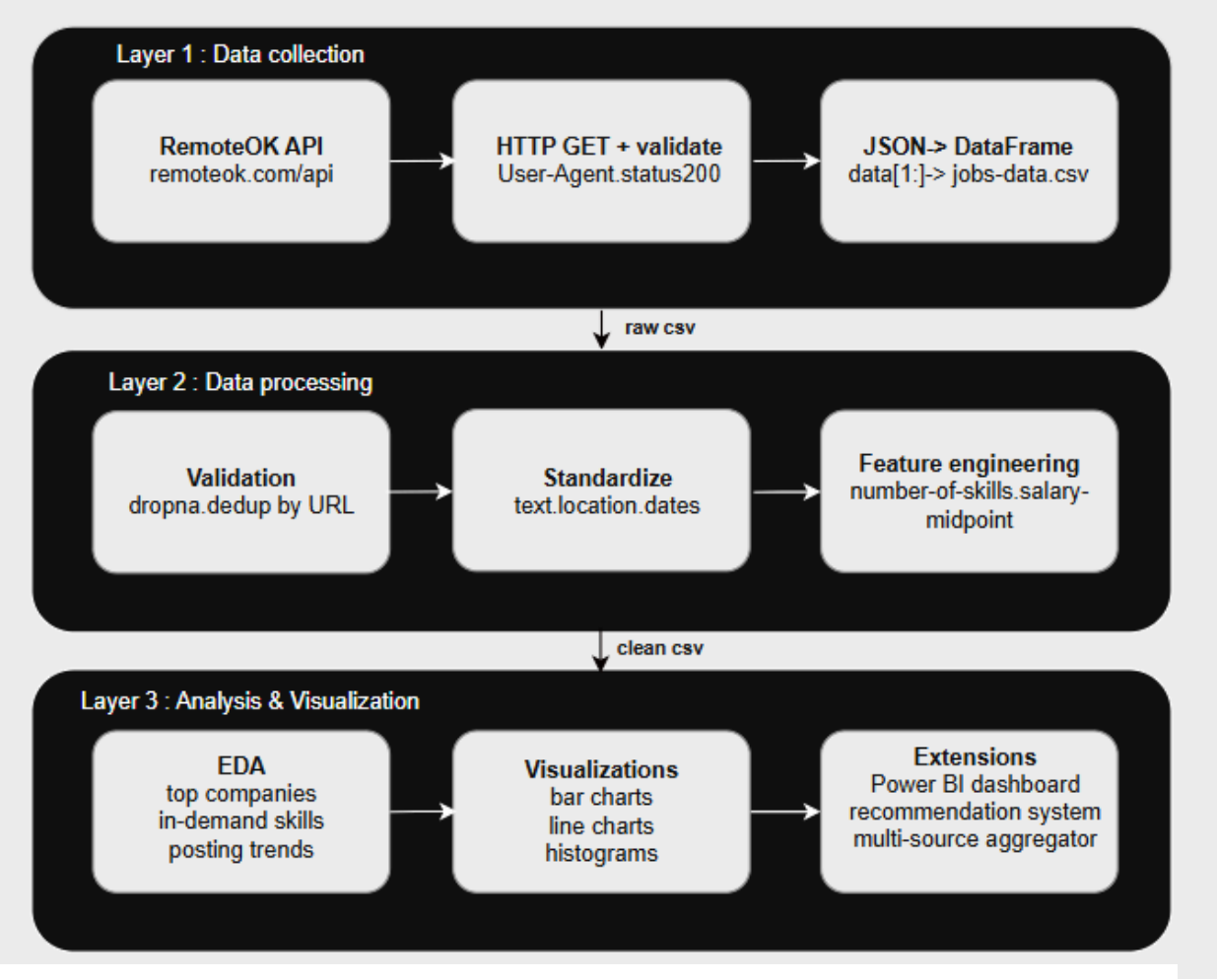
2.4. Exchange messages & data

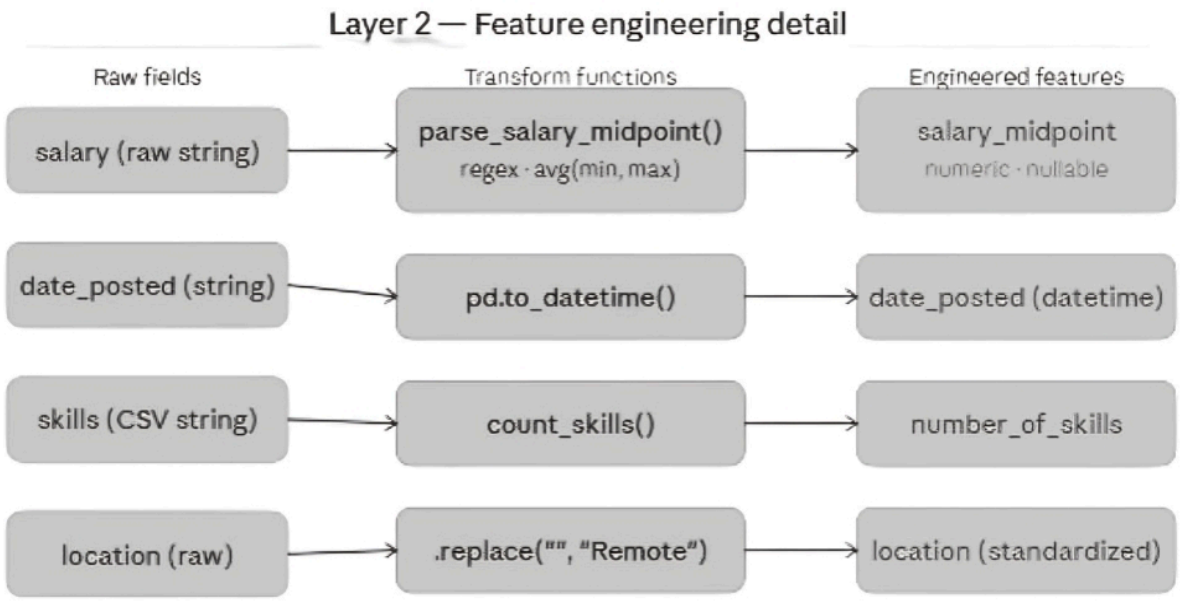
The table below details every data field that is extracted, transferred, or produced at each stage of the pipeline:

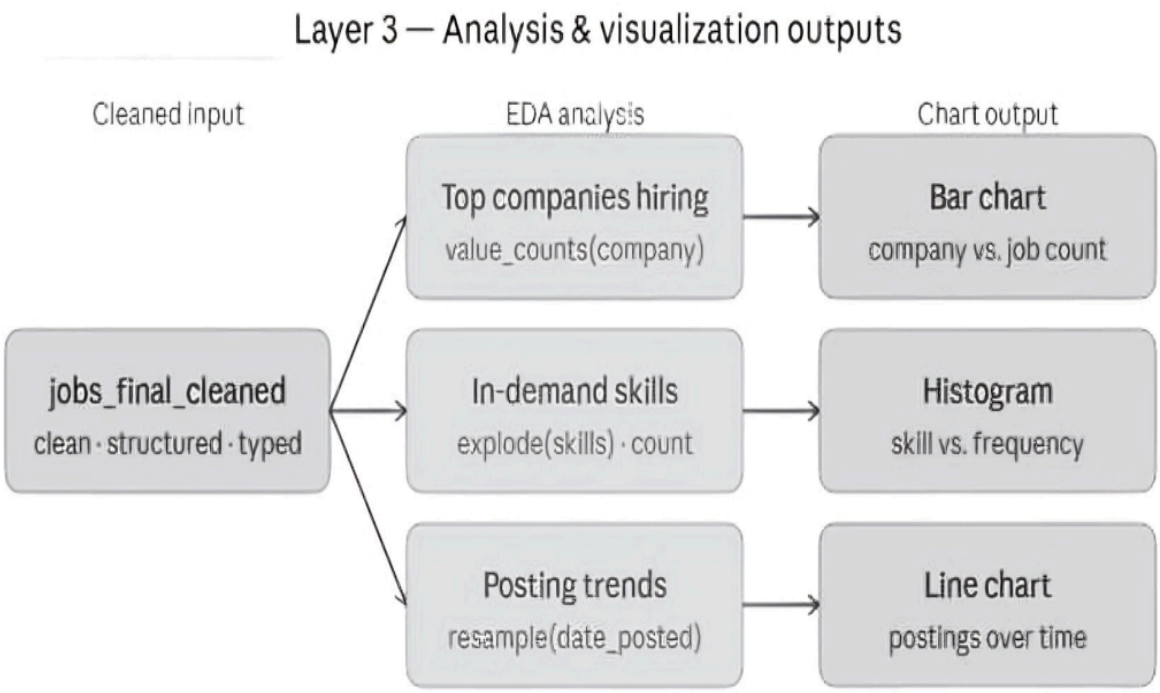
Field Name	Source	Type	Description
url	remoteok.com/api	string	Direct link to the job posting page
title	remoteok.com/api	string	Job position title (e.g. Data Analyst)
company	remoteok.com/api	string	Name of the hiring company
location	remoteok.com/api	string	Location — defaults to 'Remote' if absent
salary	remoteok.com/api	string	Raw salary string as listed on the platform
date_posted	remoteok.com/api	string	Date the job was published (ISO format)
skills (tags)	remoteok.com/api	string	List of required skills/technologies
remote	Derived	boolean	True if position is fully remote
number_of_skills	Derived	integer	Count of skills listed per job posting

2.5. System Architecture

The system is divided into three main layers:




Layer 2 — Feature engineering detail
Raw fields
Transform functions
Engineered features
salary (raw string)
parse_salary_midpoint()
regex · avg(min, max)
salary_midpoint
numeric · nullable
date_posted (string)
pd.to_datetime()
date_posted (datetime)
skills (CSV string)
count_skills()
number_of_skills
location (raw)
.replace("", "Remote")
location (standardized)


Layer 3 — Analysis & visualization outputs
Cleaned input
EDA analysis
Chart output
jobs_final_cleaned
clean · structured · typed
Top companies hiring
value_counts(company)
Bar chart
company vs. job count
In-demand skills
explode(skills) · count
Histogram
skill vs. frequency
Posting trends
resample(date_posted)
Line chart
postings over time

3. Execution Process

The execution of the project was carried out in three structured phases: **data collection, data cleaning, and data visualization**. Each phase transforms raw job data into meaningful insights.

3.1. Data Collection & Extraction

Step 1: Import libraries and set API request

The process begins by sending an HTTP GET request to the RemoteOK API endpoint:

- URL: `https://remoteok.com/api`
- A custom User-Agent header is added to simulate a real browser request.

```
import requests
import pandas as pd
import os

url = "https://remoteok.com/api"

headers = {
    "User-Agent": "Mozilla/5.0"
}
```

Step 2: Response Validation

The response status code is printed and checked:

- Ensures the API request is successful (status 200)
- If unsuccessful, the program stops using `raise_for_status()`

```
response = requests.get(url, headers=headers)
print("Status code:", response.status_code)

response.raise_for_status()
```

Step 3: JSON Parsing

The API returns data in JSON format:

- The response is converted into a Python object using `response.json()`
- The first element is metadata and is ignored

Step 4: Data Structuring

Each job entry is processed in a loop:

- Extracted fields:
  - URL
  - Job title
  - Company name
  - Location (defaulted to “Remote” if missing)
  - Salary
  - Date posted
  - Skills (converted into comma-separated string)
- Each job is stored as a dictionary inside a list

```
jobs = []

for job in data[1:]:
    jobs.append({
        "url": job.get("url"),
        "title": job.get("position"),
        "company": job.get("company"),
        "location": job.get("location") or "Remote",
        "remote": True,
        "job_type": "Remote",
        "salary": job.get("salary"),
        "date_posted": job.get("date"),
        "skills": ", ".join(job.get("tags", []))
    })
```

Step 5: DataFrame Creation

- The list of job dictionaries is converted into a Pandas DataFrame
- A preview of the dataset is printed (`df.head()`)

```
df = pd.DataFrame(jobs)

print(df.head())
print("Shape:", df.shape)
```

Step 6: Data Export

- The raw dataset is saved as:
  - `jobs_data.csv`
- Encoding is set to UTF-8 for compatibility

```
output_path = os.path.join(os.getcwd(), "jobs_data.csv")
df.to_csv(output_path, index=False, encoding="utf-8-sig")

print("CSV created here:")
print(output_path)
```

3.2. Data Cleaning & Transformation

Step 1: Data Loading

- The raw dataset `jobs_data.csv` is loaded into a DataFrame

```
import pandas as pd
import re

df = pd.read_csv("jobs_data.csv", encoding="utf-8-sig")

print("Raw shape:", df.shape)
print(df.head())
```

Step 2: Data Validation

- Rows with missing title and company are removed
- Duplicate jobs are removed based on URL

```
df.dropna(subset=["title", "company"], how="all", inplace=True)
df.drop_duplicates(subset=["url"], keep="first", inplace=True)
```

Step 3: Text Standardization

- String columns are cleaned:
  - whitespace removal
  - missing values replaced with empty strings
- Location values are standardized:
  - missing values replaced with “Remote”

```
text_columns = ["title", "company", "location", "job_type", "skills"]

for col in text_columns:
    df[col] = df[col].fillna("").astype(str).str.strip()
```

Step 4: Boolean & Missing Value Handling

- The `remote` column is converted to boolean
- Missing values are handled safely across fields

```
df["location"] = df["location"].replace("", "Remote")
df["remote"] = df["remote"].fillna(True).astype(bool)
```

Step 5: Salary Processing

- Salary values are cleaned using a custom function
- A numeric estimation (`salary_midpoint`) is extracted using regex
  - If range exists → average is calculated
  - If single value exists → used directly

```
def clean_salary(salary):
    if pd.isna(salary) or str(salary).strip() == "":
        return None
    return str(salary).strip()

df["salary"] = df["salary"].apply(clean_salary)
```

```python
def parse_salary_midpoint(salary):
    if pd.isna(salary):
        return None

    salary = str(salary)
    numbers = re.findall(r"\d+", salary.replace(",", ""))

    if len(numbers) >= 2:
        return (float(numbers[0]) + float(numbers[1])) / 2
    elif len(numbers) == 1:
        return float(numbers[0])
    else:
        return None

df["salary_midpoint"] = df["salary"].apply(parse_salary_midpoint)
```

Step 6: Date Conversion

- `date_posted` is converted into datetime format for time-based analysis

Step 7: Skills Processing

- Skills are standardized to lowercase
- A new feature is created:
    - `number_of_skills` → counts skills per job

```python
def count_skills(skills):
    if skills == "":
        return 0
    return len([s for s in skills.split(",") if s.strip() != ""])

df["number_of_skills"] = df["skills"].apply(count_skills)
```

Step 8: Final Dataset Structuring

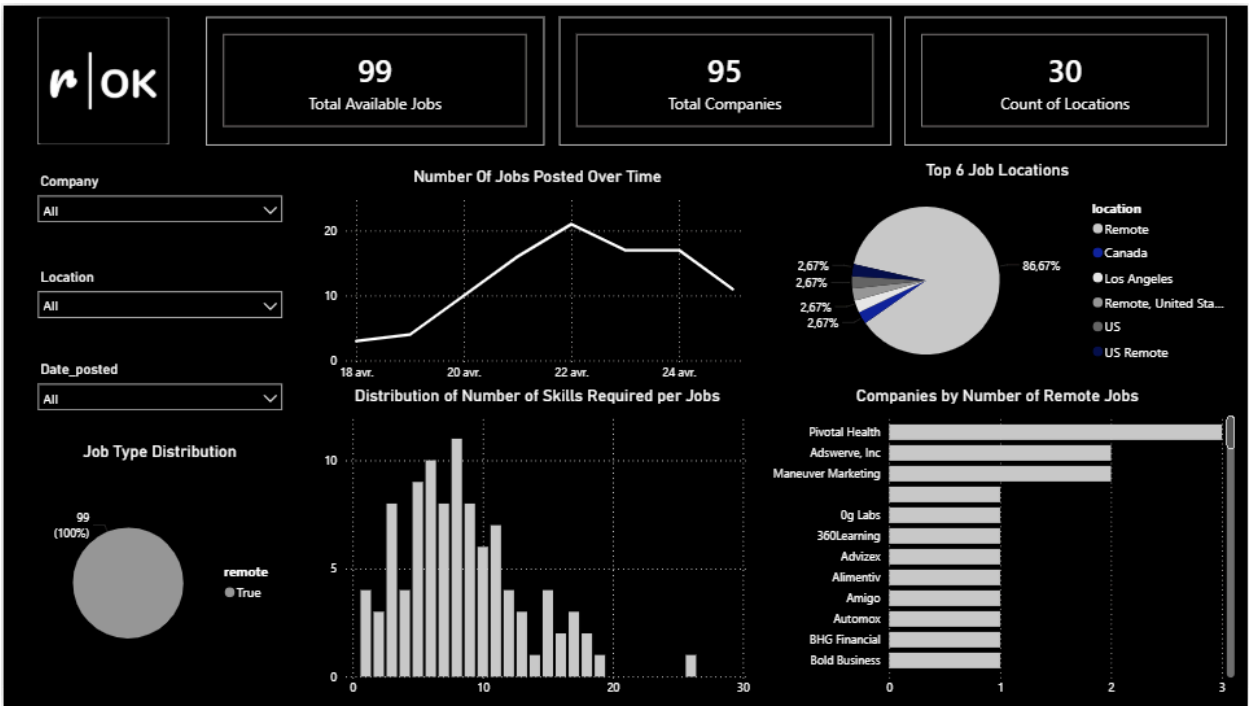
- Columns are reordered for consistency and clarity
- Clean dataset is saved as:
    - `jobs_final_cleaned.csv`

```python
df = df[
    [
        "url",
        "title",
        "company",
        "location",
        "remote",
        "job_type",
        "date_posted",
        "skills",
        "number_of_skills"
    ]
]

df.to_csv("jobs_final_cleaned.csv", index=False, encoding="utf-8-sig")

print("Cleaned shape:", df.shape)
print("jobs_final_cleaned.csv created successfully")
```

3.3. Data Visualization & Insights



Dashboard Overview – Job Market Analytics (r|OK)

This interactive dashboard provides a snapshot of the current job market data tracked through the r|OK platform. It summarizes **99 total available jobs** across **95 companies** spanning **30 distinct locations**.

Key Components:

KPI Cards (top) display the three core metrics at a glance — total jobs, total companies, and location count — offering an immediate high-level summary.

Filters (left panel) allow users to slice the data dynamically by Company, Location, and Date Posted, enabling targeted analysis across any subset of the dataset.

Number of Jobs Posted Over Time (line chart) tracks posting activity between April 18–25, revealing a clear peak around April 22 (~20 postings) followed by a decline, suggesting a burst of hiring activity mid-period.

Distribution of Skills Required per Job (histogram) shows that most job postings require between 5 and 15 skills, with the distribution peaking around 10–12 skills, indicating a moderate-to-high skill demand profile across roles.

Top 6 Job Locations (pie chart) highlights a strong concentration of remote work, with **Remote** accounting for 86.67% of all positions, followed by Canada, Los Angeles, and other locations each at 2.67%.

Job Type Distribution (donut chart) confirms that **100% of the 99 listed jobs are remote**, reinforcing the remote-first nature of this job market segment.

Companies by Number of Remote Jobs (bar chart) lists the top employers by remote job volume. Pivotal Health, Adsweve Inc., and Maneuver Marketing lead with approximately 3 postings each, while the remaining companies each contribute 1–2 remote roles.

Overall, the dashboard paints a picture of a predominantly remote, skill-intensive job market with hiring activity concentrated within a narrow time window in late April.

4. Conclusion

This project demonstrates a complete data extraction and transformation pipeline using real-world job market data.

Key outcomes:

- Successfully connected to a live job API
- Extracted structured job data without web scraping instability
- Filtered and organized data into meaningful insights
- Built a reusable workflow for job market analysis

This approach highlights a key industry principle:

> Modern data extraction is shifting from HTML scraping to API-based data engineering.

The project can be extended into:

- a job analytics dashboard
- a recommendation system
- or a multi-source job aggregator platform

— End of Report —